

AI Assisted Coding

ASSIGNMENT 19.3

Name: K. Rupaas shyni

HT No: 2303A52417

Batch: 32

Task 1 – Python to C++ Conversion

Task:

Translate a simple Python class into C++ using AI assistance.

Instructions:

- Prompt AI to convert a Python class representing a Student with attributes and methods into equivalent C++ code
- Ensure proper handling of:
 - o Constructors
 - o Data types
 - o Access specifiers
- Compile and run both versions to verify output consistency

Expected Output:

An equivalent working C++ class translated from Python.

Prompts:

1st Prompt

Write a Python program to create a Student class with attributes name and age, including a constructor and a display() method.

2nd Prompt

Convert the following Python Student class into equivalent C++ code. Ensure proper use of constructors, data types, and access specifiers.

Code:

```
lab 19.3 > 1.py > ...
1  # Write a Python program to create a Student class with attributes name and age,
2  # including a constructor and a display() method.
3  class Student:
4      def __init__(self, name, age):
5          self.name = name
6          self.age = age
7
8      def display(self):
9          print(f"Name: {self.name}, Age: {self.age}")
10
11 # Example usage
12 student = Student("Alice", 20)
13 student.display()
14
```

Output:

```
Name: Alice, Age: 20
```

Code:

```
lab 19.3 > G+ 1.cpp > ...
1  /* Convert the following Python Student class into equivalent C++ code.
2  Ensure proper use of constructors, data types, and access specifiers.
3  class Student:
4      def __init__(self, name, age):
5          self.name = name
6          self.age = age
7
8      def display(self):
9          print(f"Name: {self.name}, Age: {self.age}")
10
11 # Example usage
12 student = Student("Alice", 20)
13 student.display() */
14 #include <iostream>
15 #include <string>
16 class Student {
17 private:
18     std::string name;
19     int age;
20 public:
21     // Constructor
22     Student(const std::string& name, int age) : name(name), age(age) {}
23
24     // Method to display student information
25     void display() const {
26         std::cout << "Name: " << name << ", Age: " << age << std::endl;
27     }
28 };
29 int main() {
30     // Example usage
31     Student student("Alice", 20);
32     student.display();
33     return 0;
34 }
```

Output:

```
Name: Alice, Age: 20
```

output consistency verified.

Task 2 – Java to Python Function Conversion

Task:

Convert a Java method that checks whether a number is prime into Python.

Instructions:

- Ask AI to translate a Java `isPrime()` method into Python
- Test the function with multiple inputs
- Ensure the Python version follows Pythonic syntax and logic

Expected Output:

A correct Python function equivalent to the original Java prime-checking method.

Prompts:

1st Prompt

Write a Java program with a method `isPrime()` that checks whether a given number is prime. Include a main method to test it with at least 3 different numbers.

2nd Prompt

Convert the following Java `isPrime()` method into equivalent Python code.

Make sure to follow Pythonic syntax and test with multiple inputs including edge cases.

Java Code:

```
public class Prime {  
    public static void main(String[] args) {  
        int num1 = 7;  
        int num2 = 10;  
        int num3 = 13;  
  
        System.out.println(num1 + " is prime: " + isPrime(num1));  
        System.out.println(num2 + " is prime: " + isPrime(num2));  
        System.out.println(num3 + " is prime: " + isPrime(num3));  
    }  
}
```

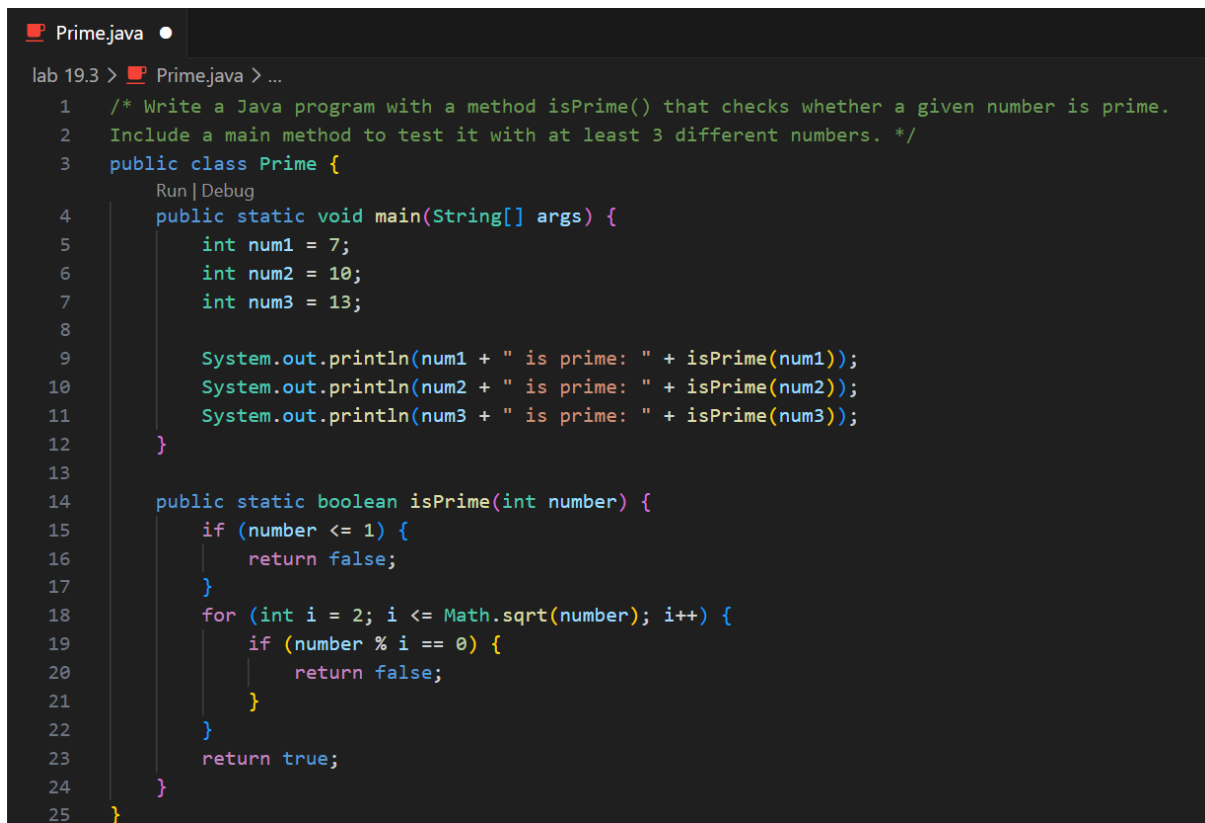
```

    }

    public static boolean isPrime(int number) {
        if (number <= 1) {
            return false;
        }
        for (int i = 2; i <= Math.sqrt(number); i++) {
            if (number % i == 0) {
                return false;
            }
        }
        return true;
    }
}

```

Code:



```

Prime.java •
lab 19.3 > Prime.java > ...
1  /* Write a Java program with a method isPrime() that checks whether a given number is prime.
2  Include a main method to test it with at least 3 different numbers. */
3  public class Prime {
4      Run | Debug
5      public static void main(String[] args) {
6          int num1 = 7;
7          int num2 = 10;
8          int num3 = 13;
9
10         System.out.println(num1 + " is prime: " + isPrime(num1));
11         System.out.println(num2 + " is prime: " + isPrime(num2));
12         System.out.println(num3 + " is prime: " + isPrime(num3));
13     }
14
15     public static boolean isPrime(int number) {
16         if (number <= 1) {
17             return false;
18         }
19         for (int i = 2; i <= Math.sqrt(number); i++) {
20             if (number % i == 0) {
21                 return false;
22             }
23         }
24         return true;
25     }
26 }

```

```
2.py
lab 19.3 > 2.py > ...

27     }
28 }
29 ...
30 import math
31 def is_prime(number):
32     if number <= 1:
33         return False
34     for i in range(2, int(math.sqrt(number)) + 1):
35         if number % i == 0:
36             return False
37     return True
38
39 if __name__ == "__main__":
40     # Test cases including edge cases
41     test_cases = [
42         -5,      # Negative number
43         0,       # Zero
44         1,       # One
45         2,       # Smallest prime
46         7,       # Prime
47         10,      # Composite
48         13,      # Prime
49         97,      # Large prime
50         100,     # Large composite
51     ]
52
53     for num in test_cases:
54         print(f"{num} is prime: {is_prime(num)}")
```

Output:

```
-5 is prime: False
0 is prime: False
1 is prime: False
2 is prime: True
7 is prime: True
10 is prime: False
13 is prime: True
97 is prime: True
100 is prime: False
```

Task 3 – Pseudocode to Python Implementation

Task:

Translate a given pseudocode algorithm into Python using AI.

Instructions:

- Provide AI with pseudocode for sorting numbers (e.g., bubble sort or selection sort)
- Ask AI to convert it into executable Python code
- Validate correctness using sample input lists

Expected Output:

A working Python program that correctly implements the pseudocode logic.

Prompt:

Pseudo code for bubble sort

for i from 0 to n-1

 for j from 0 to n-i-2

 if arr[j] > arr[j+1]

 swap arr[j] and arr[j+1]

convert pseudocode into Python with Sample input lists.

Code:

```
lab 19.3 > 3.py > ...
1  '''
2  Pseudo code for bubble sort
3  for i from 0 to n-1
4      for j from 0 to n-i-2
5          if arr[j] > arr[j+1]
6              swap arr[j] and arr[j+1]
7  '''
8  # convert pseudocode into Python with Sample input lists.
9  def bubble_sort(arr):
10     n = len(arr)
11     for i in range(n):
12         for j in range(n - i - 1):
13             if arr[j] > arr[j + 1]:
14                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
15     return arr
16  # Sample input lists
17  test_cases = [
18     [64, 34, 25, 12, 22, 11, 90],
19     [5, 1, 4, 2, 8],
20     [3, 0, 2, 5, -1],
21     [1],
22     []
23  ]
24  # Testing the bubble_sort function with multiple inputs including edge cases
25  for test in test_cases:
26     print(f"Original array: {test}")
27     sorted_array = bubble_sort(test)
28     print(f"Sorted array: {sorted_array}\n")
29
```

Output:

```
Original array: [64, 34, 25, 12, 22, 11, 90]
Sorted array: [11, 12, 22, 25, 34, 64, 90]

Original array: [5, 1, 4, 2, 8]
Sorted array: [1, 2, 4, 5, 8]

Original array: [3, 0, 2, 5, -1]
Sorted array: [-1, 0, 2, 3, 5]

Original array: [1]
Sorted array: [1]

Original array: []
Sorted array: []
```

Task 4 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

- Provide AI with a SQL query (e.g., `SELECT name, salary FROM employees WHERE salary > 50000`).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

Expected Output:

- Equivalent Pandas query that retrieves the correct subset of data.

Prompt:

Convert the following SQL query into equivalent Pandas DataFrame operations in Python.

SQL Query:

```
SELECT name, salary, department
FROM employees
WHERE salary > 50000;
```

Include:

Sample employees DataFrame with columns: name, salary, department

The equivalent Pandas query using DataFrame operations
Display the filtered results

Code:

```
lab 19.3 > 4.py > ...
1  '''
2  Convert the following SQL query into equivalent Pandas DataFrame operations in Python.
3
4  SQL Query:
5  SELECT name, salary, department
6  FROM employees
7  WHERE salary > 50000;
8
9  Include:
10 Sample employees DataFrame with columns: name, salary, department
11 The equivalent Pandas query using DataFrame operations
12 Display the filtered results
13 '''
14 import pandas as pd
15 # Sample employees DataFrame
16 employees = pd.DataFrame({
17     'name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
18     'salary': [60000, 45000, 75000, 55000, 50000],
19     'department': ['Sales', 'IT', 'Finance', 'HR', 'IT']
20 })
21
22 # Equivalent Pandas query
23 result = employees[employees['salary'] > 50000][['name', 'salary', 'department']]
24
25 # Display the filtered results
26 print(result)
27
```

Output:

	name	salary	department
0	Alice	60000	Sales
2	Charlie	75000	Finance
3	Diana	55000	HR

Task 5 – Real-Time Application: Algorithm Translation Across Languages

Scenario:

A company maintains algorithms written in different programming languages.

Examples:

- Converting a Python searching algorithm into C
- Translating a Java sorting program into JavaScript

Instructions:

- Use AI to translate a selected algorithm between two programming languages
- Execute and test both versions
- Document translation challenges such as:
 - o Syntax differences
 - o Library support
 - o Memory management

Expected Output:

Equivalent, tested code implementations in two different programming languages.

Prompt:

Prompt 1

Write a Python program that implements linear search to find an element in a list.

Prompt 2

Translate the following Python linear search algorithm into equivalent C code.

```
def linear_search(arr, target):  
    for i in range(len(arr)):  
        if arr[i] == target:  
            return i # Return the index of the target element  
    return -1 # Return -1 if the target element is not found  
# Example usage
```

```
my_list = [5, 3, 2, 8, 1]
target_element = 12
result = linear_search(my_list, target_element)
if result != -1:
    print(f"Element {target_element} found at index: {result}")
else:
    print(f"Element {target_element} not found in the list.")
```

Code:

```
5.py
lab 19.3 > 5.py > ...
1  # Write a Python program that implements linear search to find an element in a list.
2  def linear_search(arr, target):
3      for i in range(len(arr)):
4          if arr[i] == target:
5              return i # Return the index of the target element
6      return -1 # Return -1 if the target element is not found
7  # Example usage
8  my_list = [5, 3, 2, 8, 1]
9  target_element = 12
10 result = linear_search(my_list, target_element)
11 if result != -1:
12     print(f"Element {target_element} found at index: {result}")
13 else:
14     print(f"Element {target_element} not found in the list.")
15
```

```
5.c
lab 19.3 > 5.c > ...
17 #include <stdio.h>
18 int linear_search(int arr[], int size, int target) {
19     for (int i = 0; i < size; i++) {
20         if (arr[i] == target) {
21             return i; // Return the index of the target element
22         }
23     }
24     return -1; // Return -1 if the target element is not found
25 }
26 int main() {
27     int my_list[] = {5, 3, 2, 8, 1};
28     int size = sizeof(my_list) / sizeof(my_list[0]);
29     int target_element = 12;
30     int result = linear_search(my_list, size, target_element);
31     if (result != -1) {
32         printf("Element %d found at index: %d\n", target_element, result);
33     } else {
34         printf("Element %d not found in the list.\n", target_element);
35     }
36     return 0;
37 }
```

Output:

```
Element 12 not found in the list.
```

Field	Details
Algorithm	Linear Search
Languages	Python → C
Objective	Translate and test algorithm in both languages
Input	List: [5, 3, 2, 8, 1], Target: 12
Output	Not found (-1)
Python Result	Element 12 not found in the list
C Result	Element 12 not found in the list
Syntax Difference	Python simple, C strict with types
Loop Difference	range(len(arr)) vs for (int i=0; i<size; i++)
Function	Python no type, C requires return type
Memory	Python automatic, C manual
Libraries	Python none, C uses stdio.h
Output Method	print() vs printf()
Conclusion	Both give same result, Python easy, C faster